Chapter 09: Classification and Regression Trees

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Add comments to each **code** cell (no comments need to be added for Markdown).

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate. For example,

JT88

This code cell imports necessary Python packages.

In [7]:
```
import matplotlib.pyplot as plt
import mlba
import numpy as np
import pandas as pd
import scipy
%matplotlib inline
!pip install xgboost
import xgboost as xgb
print(xgb.__version__)
import xgboost as xgb
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import auc, roc_curve
from sklearn.model_selection import (GridSearchCV, RandomizedSearchCV,
                                     cross_val_score, train_test_split)
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot
```

```
Requirement already satisfied: xgboost in /opt/anaconda3/lib/python3.13/site
-packages (3.1.1)
Requirement already satisfied: numpy in /opt/anaconda3/lib/python3.13/site-p
ackages (from xgboost) (2.1.3)
Requirement already satisfied: scipy in /opt/anaconda3/lib/python3.13/site-p
ackages (from xgboost) (1.15.3)
3.1.1
```

Classification Trees

Example 1: Riding Mowers

Code for running and plotting classification tree

Figure 9.7 Tree Representation of First Split

In [8]:
```
#VB67
#Loading dataset
mower_df = mlba.load_data('RidingMowers.csv')
# use max_depth to control tree size (None = full tree)
classTree = DecisionTreeClassifier(random_state=0, max_depth=1) # max_depth
# X = all columns except the target 'Ownership'
classTree.fit(mower_df.drop(columns=['Ownership']), mower_df['Ownership'])
# Visualize the trained tree
plot_tree(classTree, feature_names=mower_df.columns[:2],
          label='root', class_names=classTree.classes_,
          impurity=False, filled=True, rounded=True)
plt.tight_layout()
#display plot
plt.show()
```

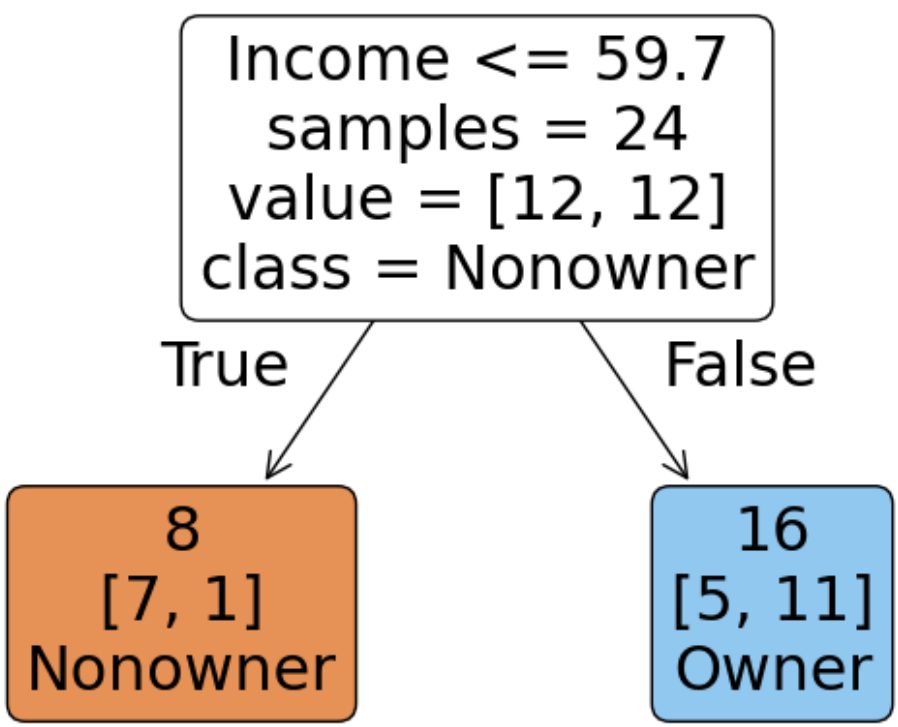


Figure 9.8 Tree Representation of First Three Splits

In [9]:
```
#VB67
# Train a simple decision tree and visualize its structure
classTree = DecisionTreeClassifier(random_state=0, max_depth=2)
classTree.fit(mower_df.drop(columns=['Ownership']), mower_df['Ownership'])
# Plot the tree to show how features are used for splitting and classificati
plt.figure(figsize=[3.5, 3], dpi=300)
plot_tree(classTree, feature_names=mower_df.columns[:2],
          label='root', class_names=classTree.classes_,
          impurity=False, filled=True, rounded=True, fontsize=7)
plt.tight_layout()
#Display plot
plt.show()
```

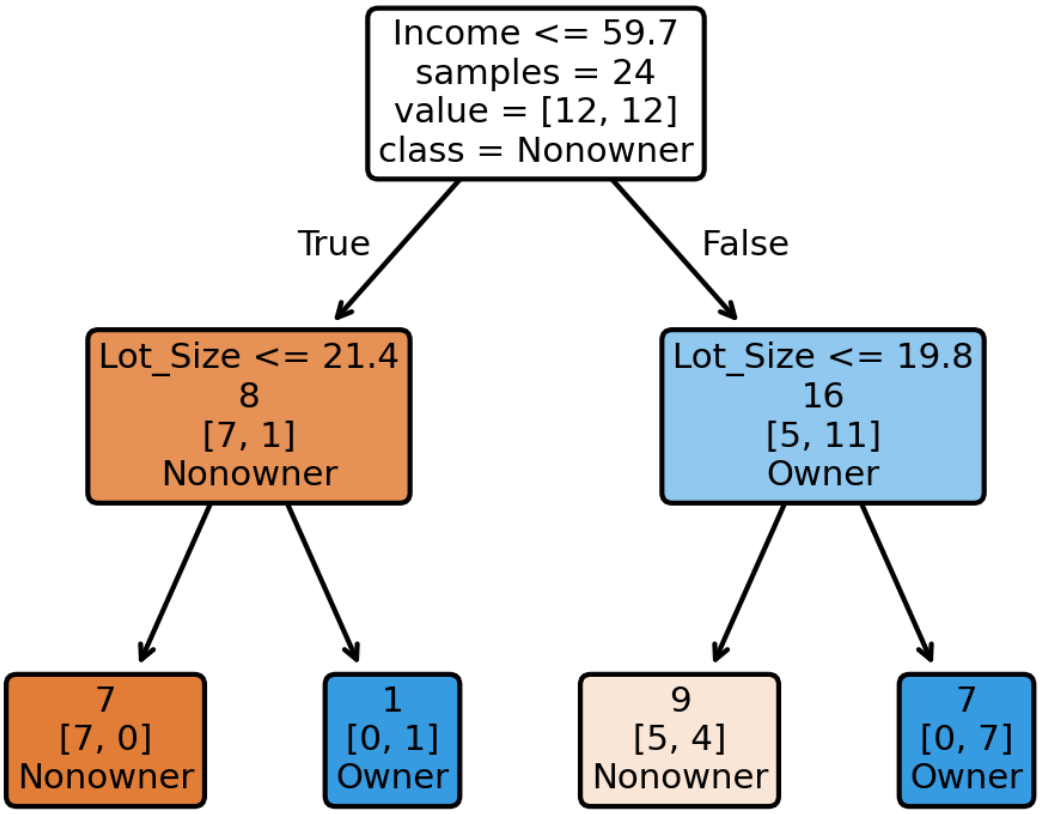
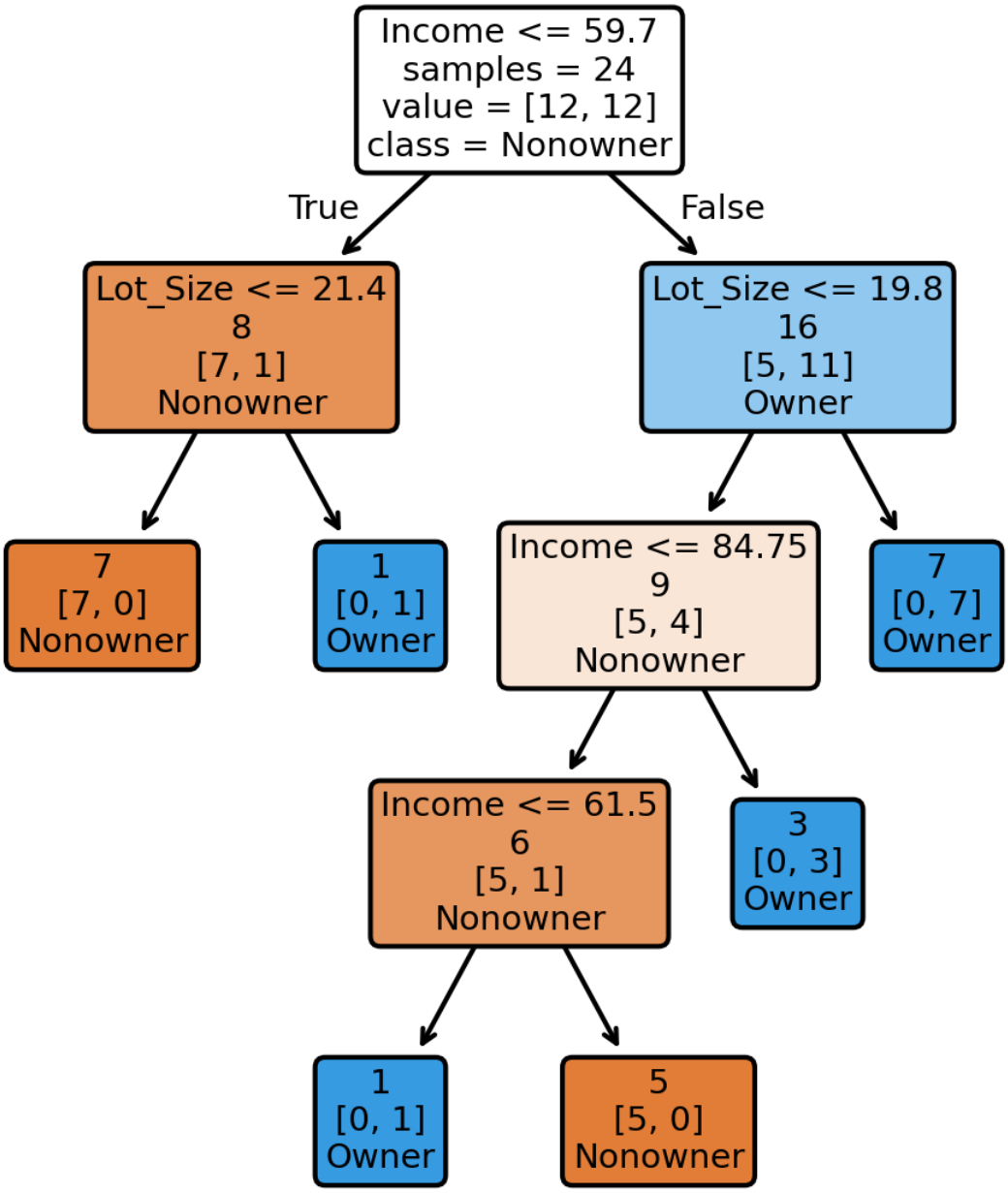


Figure 9.9 Tree Representation after All Splits (Full Grown Tree)

In [10]:
```
#VB67
# Build and visualize a full decision tree for mower ownership classificatic
classTree = DecisionTreeClassifier(random_state=0)
```

```
classTree.fit(mower_df.drop(columns=['Ownership']), mower_df['Ownership'])
# Display the decision tree structure with feature splits and class labels
plt.figure(figsize=[3.5, 4], dpi=300)
plot_tree(classTree, feature_names=mower_df.columns[:2],
          label='root', class_names=classTree.classes_,
          impurity=False, filled=True, rounded=True, fontsize=7)
plt.tight_layout()
#Display plot
plt.show()
```



We can also get a textual representation of a decision tree using the export_tree of scikit-learn. Below is the one for the fully grown tree. The first line, for example, shows that the decision at the first split is Income <= 59.7. The third line is a terminal node that

is reached for the conditions Income <= 59.7 and Lot_Size <= 21.40. The output gives the distribution of the classes as weights, here 7 and 0 records, respectively, and the class predicted by the terminal node, here Owner.

In [11]:
```
#VB67
from sklearn.tree import export_text
print(export_text(classTree, feature_names=mower_df.columns[:2], show_weight
```

```
|--- Income <= 59.70
|   |--- Lot_Size <= 21.40
|   |   |--- weights: [7.00, 0.00] class: Nonowner
|   |--- Lot_Size >  21.40
|   |   |--- weights: [0.00, 1.00] class: Owner
|--- Income >  59.70
|   |--- Lot_Size <= 19.80
|   |   |--- Income <= 84.75
|   |   |   |--- Income <= 61.50
|   |   |   |   |--- weights: [0.00, 1.00] class: Owner
|   |   |   |--- Income >  61.50
|   |   |   |   |--- weights: [5.00, 0.00] class: Nonowner
|   |   |--- Income >  84.75
|   |   |   |--- weights: [0.00, 3.00] class: Owner
|   |--- Lot_Size >  19.80
|   |   |--- weights: [0.00, 7.00] class: Owner
```

Example 2: Acceptance of Personal Loan

Figure 9.10 A Full Tree for the Loan Acceptance Data Using the Training Set

In [12]:
```
#VB67
# Load Universal Bank dataset, remove irrelevant columns, and split into tra
bank_df = mlba.load_data('UniversalBank.csv')
bank_df = bank_df.drop(columns=['ID', 'ZIP Code'])

X = bank_df.drop(columns=['Personal Loan'])
y = bank_df['Personal Loan']
train_X, holdout_X, train_y, holdout_y = train_test_split(X, y, test_size=0.
# Train a full decision tree on the training set (no max_depth) to classify
fullClassTree = DecisionTreeClassifier(random_state=1) # No max_depth specif
fullClassTree.fit(train_X, train_y)
# Visualize the full decision tree showing all feature splits and predicted
plt.figure(figsize=[22, 13], dpi=300)
plot_tree(fullClassTree, feature_names=train_X.columns, label='root',
          impurity=False, filled=True, rounded=True, fontsize=10)
plt.tight_layout()
#Display plot
plt.show()
```

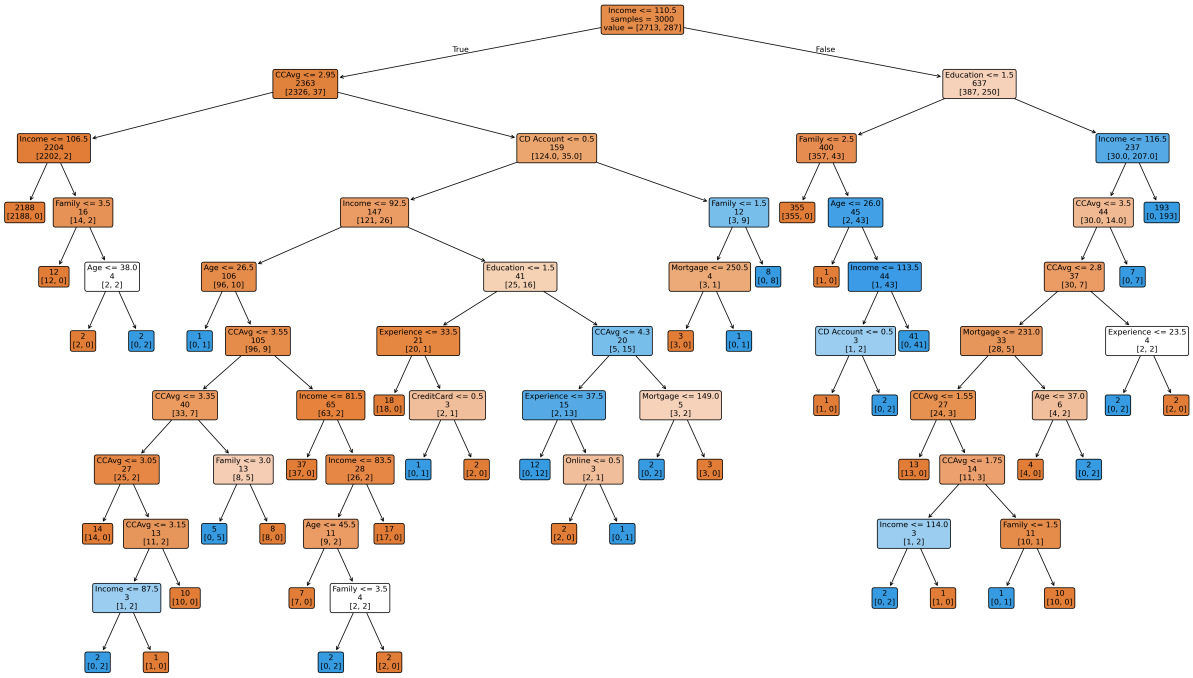


Table 9.3 Confusion Matrices and Accuracy for the Full Classification Tree, on the Training and Holdout Sets of the Personal Loan Data

In [13]:
```
#VB67
# Evaluate the performance of the full decision tree on both training and ho
# Shows metrics like accuracy, precision, recall, and F1-score
mlba.classificationSummary(y_true=train_y, y_pred=fullClassTree.predict(trai
mlba.classificationSummary(y_true=holdout_y, y_pred=fullClassTree.predict(ho
```

```
Confusion Matrix (Accuracy 1.0000)

       Prediction
Actual    0    1
     0 2713    0
     1    0  287
Confusion Matrix (Accuracy 0.9790)

       Prediction
Actual    0    1
     0 1790   17
     1   25  168
```

Table 9.4 Accuracy of the Full Classification Tree on Five Validation Folds Using 5-fold Cross-validation

In [14]:
```
#VB67
# Perform 5-fold cross-validation to evaluate the decision tree on the train
treeClassifier = DecisionTreeClassifier(random_state=1)
```

```
scores = cross_val_score(treeClassifier, train_X, train_y, cv=5)
# Prints the accuracy for each fold to assess model stability
print('Accuracy scores of each fold: ', [f'{acc:.3f}' for acc in scores])
```

```
Accuracy scores of each fold:  ['0.988', '0.973', '0.993', '0.982', '0.993']
```

Avoiding Overfitting - Stopping Tree Grow

One can think of different criteria for stopping the tree growth before it starts overfitting the data. Examples are tree depth (i.e., number of splits), minimum number of records in a terminal node, and minimum reduction in impurity. In Python's DecisionTreeClassifier(), we can control the depth of the tree, the minimum number of records in a node needed in order to split, the minimum number of records in a terminal node, etc. The problem is that it is not simple to determine what is a good stopping point using such rules.

Let us return to the personal loan example, this time restricting tree depth, the minimum number of records in a node required for splitting, and the minimum impurity decrease required. The code and resulting tree are shown in Figure 9.12. The resulting tree has five terminal nodes, where two are pure but three are not. The smallest terminal node has 44 records.

Figure 9.12 Smaller Classification Tree for the Loan Acceptance Data Using the Training Set

In [15]:
```
#VB67
# Train a pruned decision tree with constraints to prevent overfitting
# max_depth, min_samples_split, and min_impurity_decrease control tree size
smallClassTree = DecisionTreeClassifier(max_depth=30, min_samples_split=20,
                        min_impurity_decrease=0.01, random_state=1) # pruned
smallClassTree.fit(train_X, train_y)

# Visualize the pruned decision tree showing feature splits and predicted cl
plot_tree(smallClassTree, feature_names=train_X.columns, label='root',
          impurity=False, filled=True, rounded=True, fontsize=10)
plt.tight_layout()
#Display plot
plt.show()
```

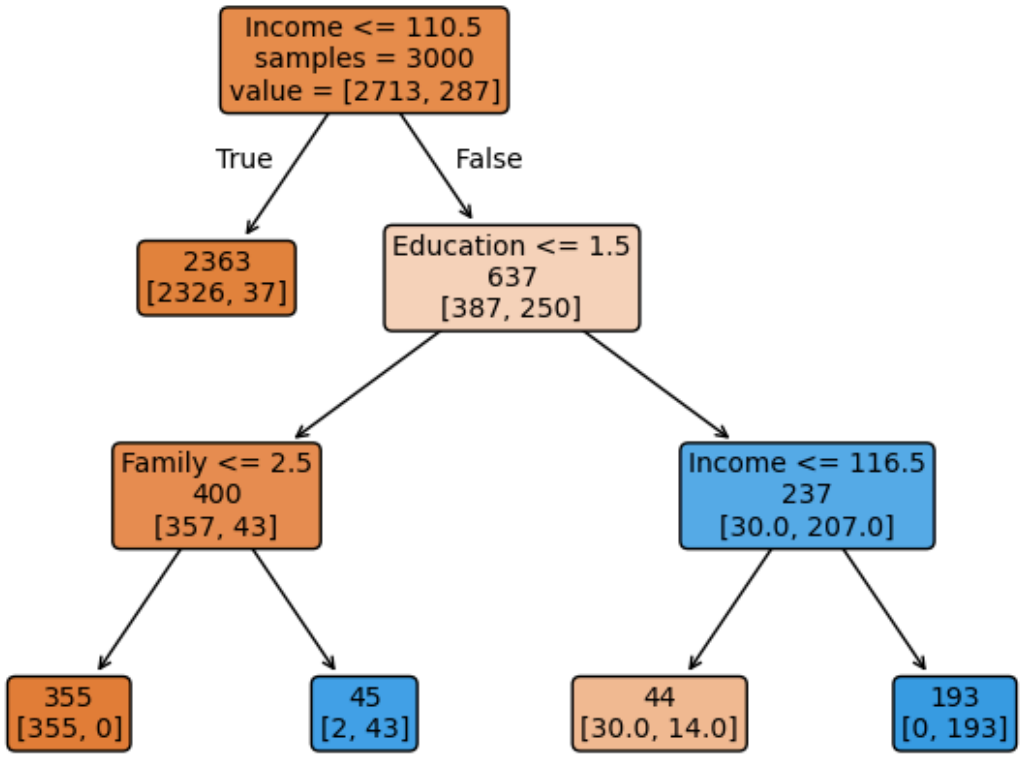


Table 9.5 Confusion Matrices and Accuracy for the Small Classification Tree, on the Training and Holdout Sets of the Personal Loan Data

Compared to the full grown tree, we see that the full tree has higher training accuracy: it is 100% accurate in classifying the training data, which means it has completely pure terminal nodes. The smaller tree achieves the same holdout performance, but importantly, it has a smaller gap in performance between training and holdout, whereas the full tree's training-vs.-holdout performance gap is bigger. The main reason is that the full-grown tree overfits the training data (to perfect accuracy!).

In [16]:
```
#VB67
# Evaluate the pruned decision tree on both training and holdout sets
# Provides metrics like accuracy, precision, recall, and F1-score to check p
mlba.classificationSummary(y_true=train_y, y_pred=smallClassTree.predict(tra
mlba.classificationSummary(y_true=holdout_y, y_pred=smallClassTree.predict(h
```

```
Confusion Matrix (Accuracy 0.9823)

       Prediction
Actual    0    1
     0 2711    2
     1   51  236
Confusion Matrix (Accuracy 0.9770)

       Prediction
Actual    0    1
     0 1804    3
     1   43  150
```

Fine-Tuning Tree Parameters

Table 9.6 Exhaustive Grid Search to Fine-tune Method Parameters

In Python, an exhaustive grid search of this type can be achieved using GridSearchCV(). The resulting parameter combination is max_depth 5, min_impurity_decrease 0.0003, and min_samples_split 20.

In [17]:
```
#VB67
# Start with an initial guess for parameters
param_grid = {
    'max_depth': [10, 20, 30, 40],
    'min_samples_split': [20, 40, 60, 80, 100],
    'min_impurity_decrease': [0, 0.0005, 0.001, 0.005, 0.01],
}
gridSearch = GridSearchCV(DecisionTreeClassifier(random_state=1), param_gric
                          n_jobs=-1)  # n_jobs=-1 will utilize all available
gridSearch.fit(train_X, train_y)
print(f'Initial score: {gridSearch.best_score_:.4f}')
print('Initial parameters: ', gridSearch.best_params_)

# Adapt grid based on result from initial grid search
param_grid = {
    'max_depth': list(range(2, 16)),  # 14 values
    'min_samples_split': list(range(10, 22)), # 11 values
    'min_impurity_decrease': [0.0003, 0.0005, 0.0007], # 3 values
}
gridSearch = GridSearchCV(DecisionTreeClassifier(random_state=1), param_gric
                          n_jobs=-1)
gridSearch.fit(train_X, train_y)
print(f'Improved score: {gridSearch.best_score_:.4f}')
print('Improved parameters: ', gridSearch.best_params_)

bestClassTree = gridSearch.best_estimator_
```

```
Initial score: 0.9877
Initial parameters:  {'max_depth': 10, 'min_impurity_decrease': 0.0005, 'min
_samples_split': 20}
Improved score: 0.9883
Improved parameters:  {'max_depth': 5, 'min_impurity_decrease': 0.0003, 'min
_samples_split': 20}
```

Evaluating performance

In [18]:
```
#VB67
# fine-tuned tree: training
mlba.classificationSummary(y_true=train_y, y_pred=bestClassTree.predict(trai
```

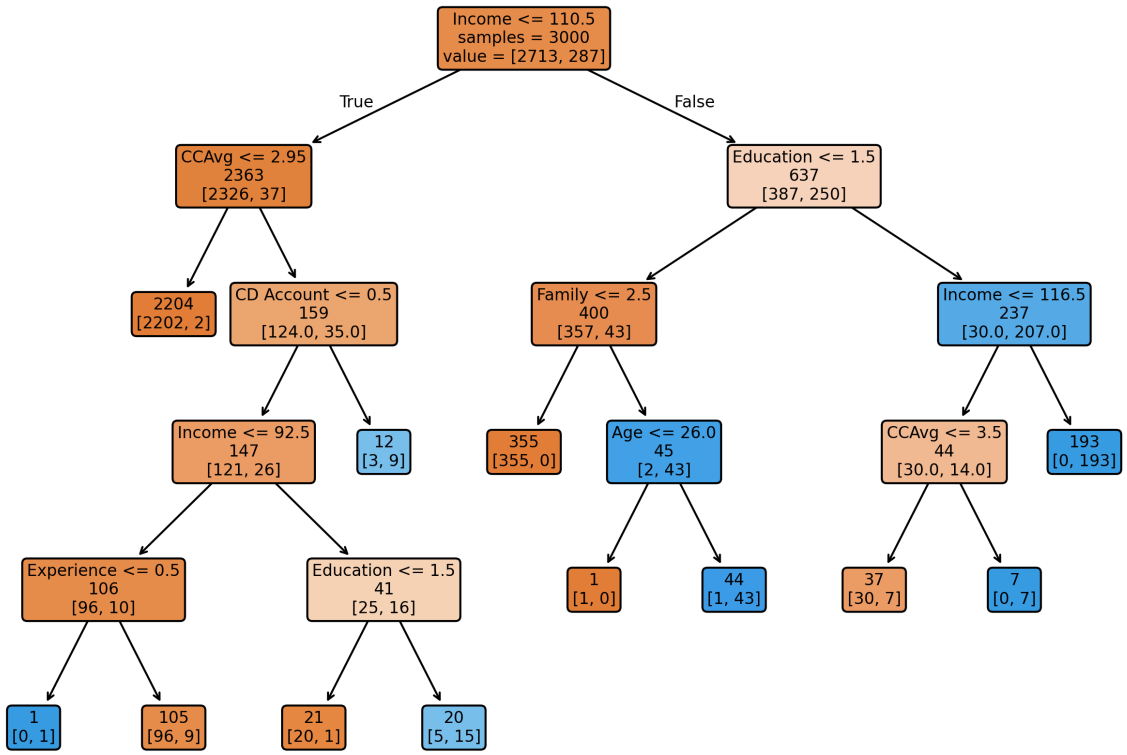
```
Confusion Matrix (Accuracy 0.9907)

       Prediction
Actual    0    1
     0 2704    9
     1   19  268
```

In [19]:
```
#VB67
# fine-tuned tree: holdout
mlba.classificationSummary(y_true=holdout_y, y_pred=bestClassTree.predict(ho
```

```
Confusion Matrix (Accuracy 0.9825)

       Prediction
Actual    0    1
     0 1793   14
     1   21  172
```

Plotting tree

In [20]:
```
#VB67
plt.figure(figsize=[8.5, 6], dpi=300)
plot_tree(bestClassTree, feature_names=train_X.columns, label='root',
          impurity=False, filled=True, rounded=True, fontsize=8)
plt.tight_layout()
#display plot
plt.show()
```



Regression Trees

Example 3: Toyota Corolla

The goal is to find a predictive model of price as a function of 10 predictors (including mileage, horsepower, number of doors, etc.). A regression tree for these data was built using a training set of 600 records.

Table 9.7 Train and Evaluate a Fine-tuned Regression Tree

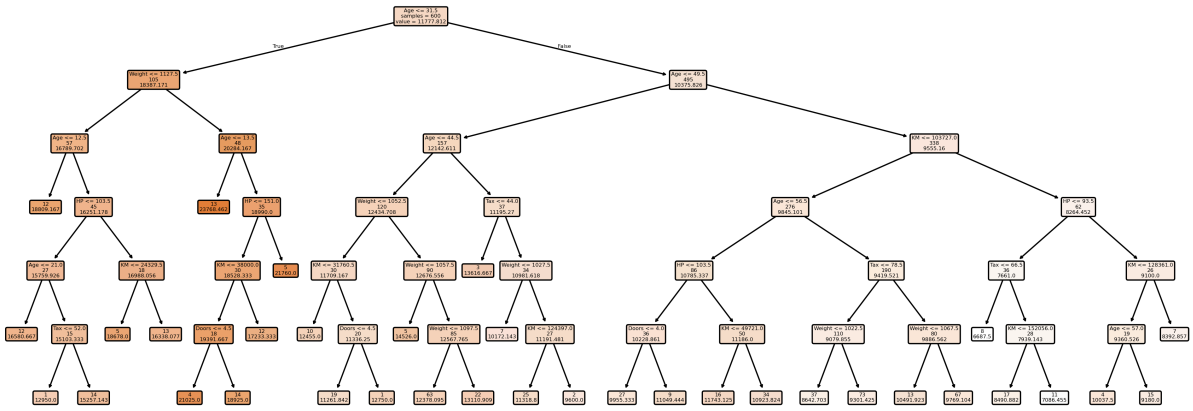
In [21]:
```
#VB67
# Load Toyota Corolla dataset, select predictors and outcome, and split into
toyotaCorolla_df = mlba.load_data('ToyotaCorolla.csv').iloc[:1000,:]
toyotaCorolla_df = toyotaCorolla_df.rename(columns={'Age_08_04': 'Age', 'Qua

predictors = ['Age', 'KM', 'Fuel_Type', 'HP', 'Met_Color', 'Automatic', 'CC'
              'Doors', 'Tax', 'Weight']
outcome = 'Price'

X = pd.get_dummies(toyotaCorolla_df[predictors], drop_first=True)
y = toyotaCorolla_df[outcome]

train_X, holdout_X, train_y, holdout_y = train_test_split(X, y, test_size=0.
```

```
# user randomized search to find optimized tree
# define a dictionary of hyperparameters to search
param_dist = {
    'max_depth': list(range(1, 25)),
    'min_impurity_decrease': scipy.stats.expon(scale=1),
    'min_samples_split': list(range(2, 50)),
}

randSearch = RandomizedSearchCV(DecisionTreeRegressor(), param_dist, cv=5,
                                n_jobs=-1, n_iter=100, random_state=1)
randSearch.fit(train_X, train_y)
print('Best parameters: ', randSearch.best_params_)

# Retrieve the best-performing regressor and evaluate performance on trainir
regTree = randSearch.best_estimator_ # retrieve the best-performing model fr

print(pd.DataFrame({
    'training': mlba.regressionMetrics(y_true=train_y, y_pred=regTree.predic
    'holdout': mlba.regressionMetrics(y_true=holdout_y, y_pred=regTree.predi
}).transpose().round(3))
```

```
Best parameters:  {'max_depth': 6, 'min_impurity_decrease': np.float64(0.601
4829269986204), 'min_samples_split': 14}
             ME      RMSE       MAE    MPE   MAPE
training   0.00  1054.903   764.644 -0.803  6.809
holdout   55.91  1553.718  1023.344 -1.338  9.213
```

In [22]:

```
#VB67
# Visualize the optimized decision tree regressor for Toyota Corolla prices
# Shows how feature splits determine predicted Price values
plt.figure(figsize=[13, 5], dpi=300)
plot_tree(regTree, feature_names=train_X.columns, label='root',
          impurity=False, filled=True, rounded=True, fontsize=4)
plt.tight_layout()
plt.show()
```



Random Forests

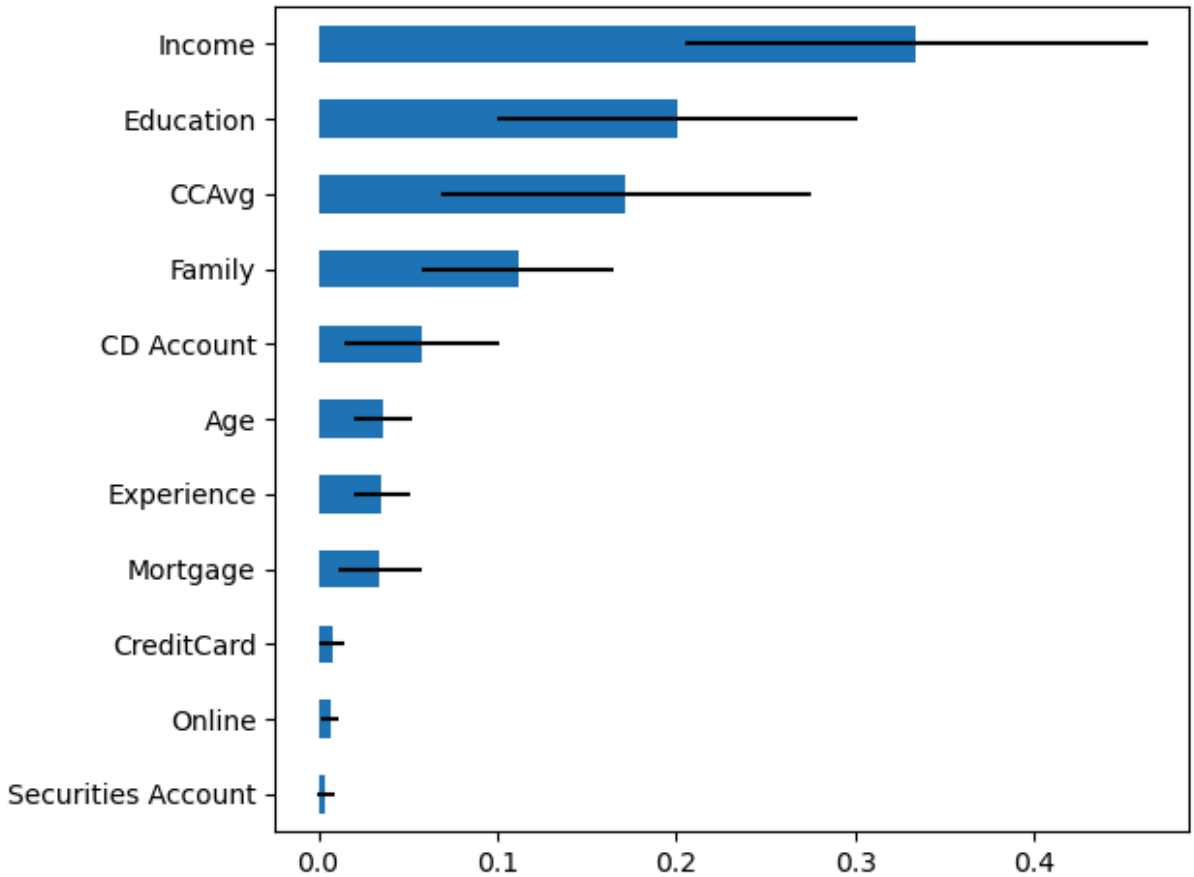
Table 9.8 Random Forest (Personal Loan Example)

In [23]:
```
#VB67
#loading dataset
bank_df = mlba.load_data('UniversalBank.csv')
bank_df = bank_df.drop(columns=['ID', 'ZIP Code']) # drop columns not useful

X = bank_df.drop(columns=['Personal Loan'])
y = bank_df['Personal Loan']
train_X, holdout_X, train_y, holdout_y = train_test_split(X, y, test_size=0.

# Initialize a Random Forest classifier: build 500 decision trees in the for
rf = RandomForestClassifier(n_estimators=500, random_state=1)
rf.fit(train_X, train_y)

# Get the importance score for each feature (impurity reduction) (average im
importances = rf.feature_importances_

# Compute the standard deviation of feature importances across all trees in
std = np.std([tree.feature_importances_ for tree in rf.estimators_], axis=0)

df = pd.DataFrame({'feature': train_X.columns, 'importance': importances, 's
df = df.sort_values('importance')
print(df)
```

```
               feature  importance       std
7   Securities Account    0.003882  0.004752
9               Online    0.006406  0.005354
10          CreditCard    0.007642  0.006951
6             Mortgage    0.034209  0.023448
1           Experience    0.035575  0.016021
0                  Age    0.036275  0.015869
8           CD Account    0.057934  0.043198
3               Family    0.111388  0.053174
4                CCAvg    0.171907  0.102980
5            Education    0.200715  0.100976
2               Income    0.334067  0.129097
```

Figure 9.16 Variable Importance Plot from Random Forest

In [24]:
```
#VB67
# Create a horizontal bar plot of features with error bars
# 'xerr' shows standard deviation, legend is hidden, and y-axis label is rem
ax = df.plot(kind='barh', xerr='std', x='feature', legend=False)
ax.set_ylabel('')
plt.tight_layout()
#Display plot
plt.show()
```



In [25]:
```
#VB67
# confusion matrix for holdout set
mlba.classificationSummary(y_true=holdout_y, y_pred=rf.predict(holdout_X))
```

```
Confusion Matrix (Accuracy 0.9820)

       Prediction
Actual    0    1
     0 1803    4
     1   32  161
```

Boosted Trees

Table 9.9 Boosted Tree: Confusion Matrix for the Holdout Set

In [28]:
```
#VB67
# Train an XGBoost classifier on the training set
boost = xgb.XGBClassifier()
boost.fit(train_X, train_y)

# Evaluate performance on the holdout set using classification metrics
mlba.classificationSummary(y_true=holdout_y, y_pred=boost.predict(holdout_X)
```

```
Confusion Matrix (Accuracy 0.9840)

       Prediction
Actual    0    1
     0 1800    7
     1   25  168
```

Figure 9.17 ROC Curves Comparing Performance of Boosted Tree, Random Forest, and Classification Tree on Holdout Set. Right Plot Zooms in to High Sensitivity Values.

The ROC curves show that both the Random Forest and XGBoost models outperform the single decision tree overall, as reflected in their higher AUC values. In the top-left corner of the ROC space — representing both high specificity (low false positive rate) and high sensitivity (true positive rate close to 1) — Random Forest slightly outperforms XGBoost at the very lowest false positive rates, although their performance is very close across this range. The single decision tree performs worse in both overall and high-specificity/high-sensitivity performance.

In [29]:

```
#VB67
# Define a function to plot ROC curves with full-scale and zoomed-in views
def plot_roc_curve(model, X, y, model_name, pos_label, *, color, axes):
    fpr, tpr, _ = roc_curve(y, model.predict_proba(X)[:, 1], pos_label=pos_l
    roc_auc = auc(fpr, tpr)
    # Plot the ROC curve on all provided axes
    for ax in axes:
        ax.plot(fpr, tpr, color=color, label=f'{model_name} (AUC = {roc_auc:
        ax.plot([0, 1], [0, 1], color='grey', linestyle='--')
        ax.set_xlabel('False positive rate (1 - specificity)')
        ax.set_ylabel('True positive rate (sensitivity)')
        ax.legend(loc="lower right")
    # Set axis limits for full-scale and zoomed-in views
    axes[0].set_xlim([-0.02, 1.02])
    axes[0].set_ylim([-0.02, 1.02])
    axes[1].set_xlim([-0.004, 0.2])
    axes[1].set_ylim([0.8, 1.004])
    return ax

# Create subplots: two side-by-side axes for full-scale and zoomed-in ROC pl
fig, axes = plt.subplots(ncols=2, figsize=[10, 5])
# Plot ROC curve for the Decision Tree model on the holdout set
plot_roc_curve(bestClassTree, holdout_X, holdout_y, 'Decision tree', 1, colo
               axes=axes)
# Plot ROC curve for the Random Forest model on the holdout set
plot_roc_curve(rf, holdout_X, holdout_y, 'Random forest', 1, color='blue', a
# Plot ROC curve for the XGBoost model on the holdout set
plot_roc_curve(boost, holdout_X, holdout_y, 'xgboost', 1, color='red', axes=
# Adjust layout to prevent overlap and display the plots
plt.tight_layout()
```

```
#display plot
plt.show()
```

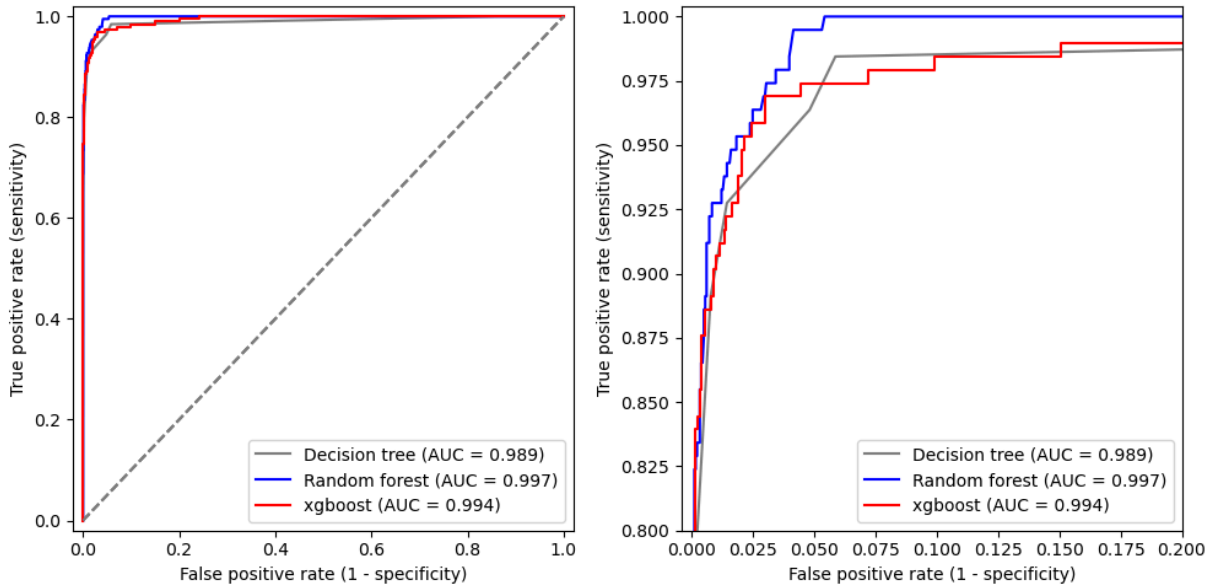
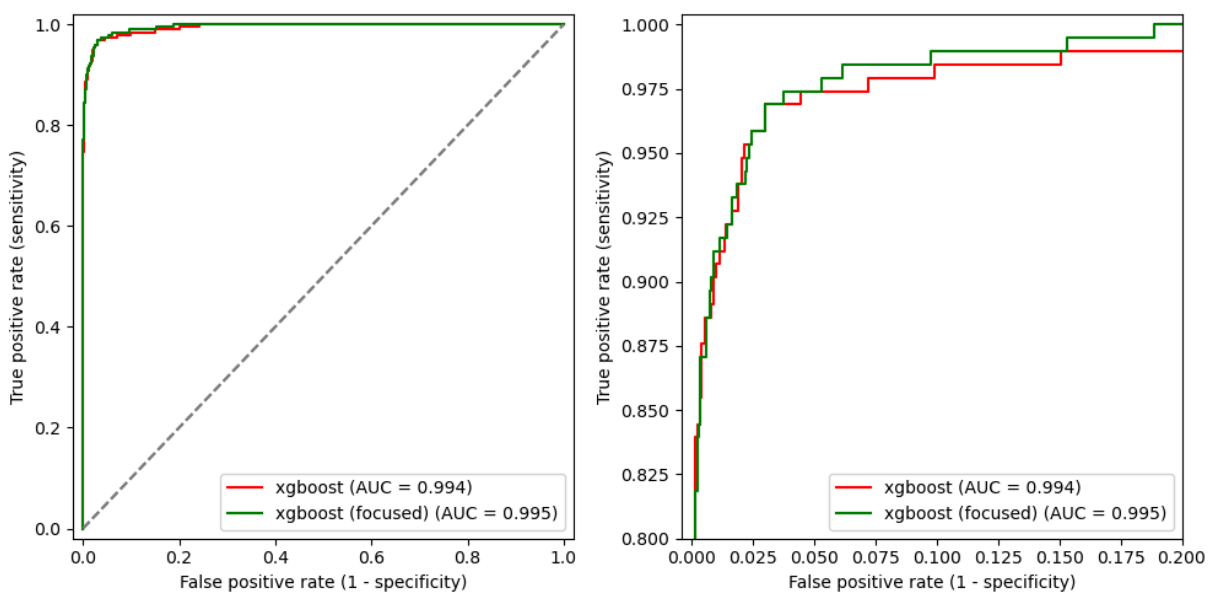


Figure 9.18 ROC Curves of Boosted Tree and Focused Boosted Tree

"scale_pos_weight" in XGBoost adjusts the training loss so that mistakes on the positive class carry more weight, which can be useful for imbalanced datasets where positives are rare. In the ROC comparison, both the original and focused models have the same overall AUC, indicating similar ranking ability across thresholds. In the zoomed-in high-specificity/high-sensitivity region, the green curve shows slightly higher sensitivity at some low false positive rates, but differences are minimal. It means that adding class weighting did not significantly change the model's overall ranking ability, but it made a slight adjustment in how the model balances sensitivity (True Positive) and specificity (True Negative) in certain parts of the curve.

In [30]:
```
#VB67
# Make a boosted tree that "cares" more about the positive class (1)
boost_focused = xgb.XGBClassifier(scale_pos_weight=10)
boost_focused.fit(train_X, train_y)
# Evaluate the focused boosted tree on the holdout set using classification
mlba.classificationSummary(y_true=holdout_y, y_pred=boost_focused.predict(ho

# Create subplots for ROC curves: full-scale and zoomed-in views
fig, axes = plt.subplots(ncols=2, figsize=[10, 5])
# Plot ROC curve for the original XGBoost model
plot_roc_curve(boost, holdout_X, holdout_y, 'xgboost', 1, color='red', axes=
# Plot ROC curve for the positive-class-focused XGBoost model
plot_roc_curve(boost_focused, holdout_X, holdout_y, 'xgboost (focused)', 1,
               color='green', axes=axes)
# Adjust layout and display the ROC plots for comparison
plt.tight_layout()
plt.show()
```

```
Confusion Matrix (Accuracy 0.9830)

       Prediction
Actual    0    1
     0 1796   11
     1   23  170
```



In [ ]: